

一、什么是软件体系结构

答：构建，连接件，约束。

构建：可以是一段代码，如程序模块；也可以是独立的程序，如数据库的 SQL 服务器。

连接件：表示构件之间相互作用，可以是过程调用，管道和消息。

约束：是构建连接时的条件。

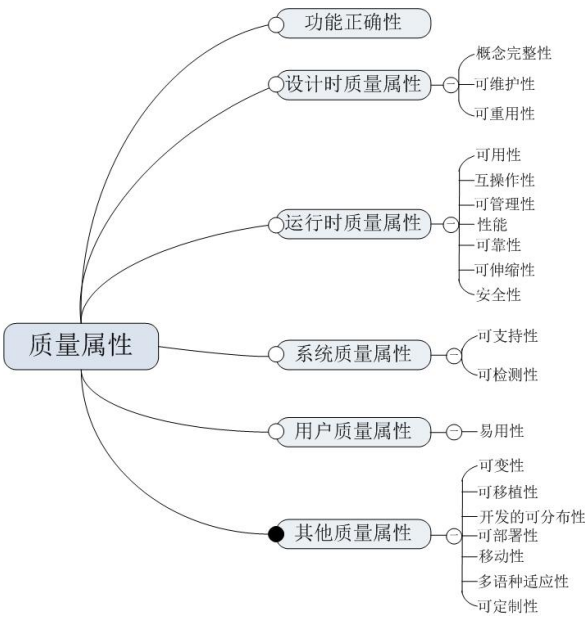
二、质量属性

2.1 理解质量属性

1. 软件质量属性概念

软件质量属性指的是一个系统的可度量、可测试的属性，这些属性会影响到系统的运行时行为、系统设计方式以及用户的体验等。

2. 质量属性分类



3. 场景的六个部分

质量属性场景分为一般场景和具体场景。

质量属性通过场景来描述，包含以下 6 部分组成。

- (1) 刺激源。某个生成该刺激的实体。
- (2) 刺激。当到达时引起系统进行响应的条件
- (3) 环境。刺激到达时，系统所处的状态，如系统可能处于过载，或者正在运行或处于其他情况；或指该刺激在系统的某些条件内发生。
- (4) 制品。被刺激的部分。可能是整个系统，也可能是系统的一部分。
- (5) 响应。表示系统在刺激到达后所采取的行为或措施。
- (6) 响应度量。当响应发生时，我们以某种方式对其进行度量，便于我们对需求进行测试。包括响应时间、响应的准确性、响应所带来的影响等。

场景元素	可能的值	示例
刺激源	开发人员，设计人员，架构师	开发人员
刺激	重构、迁移、新成员、小组沟通	在实时定位系统中，对系统某一定位算法功能进行重构
制品	组件、代码、文档、系统	代码，设计文档
环境	在设计、开发、编译、部署或运行时	系统正常运行
响应	开发人员根据一致的概念重构代码；将系统迁移到新的平台运行；新成员遵循编码规范进行编码；建立协调和统一的沟通方式	移除原有算法，将重构后代码集成到系统中
响应度量	开发人员重构的时间；系统迁移所需工作量等	系统重构所修改的代码行数

2.2 设计时质量属性

1. 概念完整性

概念完整性指架构设计的一致性，定义了整体设计的一致性和连贯性。保证了在整个系统中所有的概念及其相互关系始终保持一致。整个架构中同一件事情按照同样的方式被执行。

2. 可维护性

可维护性是指系统可以承受一定程度修改的能力。可维护性还包括为了升级让某个操作出现中断或移除某个操作之后需要花费多少时间才能让系统恢复正常的的能力。

3. 可重用性（策略模式）

可重用性定义了组件和子系统在其他应用程序或应用场景中进行重用的能力。可重用性最小化了组件的重复性及实现时间。确定各种组件之间的公共特性是构建用于大型系统中的小型可重用组件的第一步。

2.3 运行时质量属性

1. *可用性

可用性定义了系统正常运转并发挥功能的能力，它受系统故障及其相关后果影响。

可用性：累计工作时间/总时间

$$\alpha = \frac{MTBF}{MTBF + MTTR}$$

MTBF(failure)：平均故障间隔时间，累计工作时间/故障次数

MTTR(repair)：平均修复时间，总宕机时间/故障次数

2. *可靠性

广义的可靠性：一切旨在避免、减少、处理、度量软件故障的分析、设计、测试方法、技术和实践活动。

狭义的可靠性：软件无失效运行的定量度量。

$$R(t) = e^{-(t/\theta)}$$

t 是总时间,θ取 MTBF（平均故障间隔时间）

可靠性通常低于可用性。

3. 互操作性

互操作性是指描述两个或多个软件构件协作的能力，即不同来源的构件能相互协调、通信，共同完成更复杂的功能的能力。一个可互操作的系统使得信息可以在内部和外部进行更方便的交换和重用。互操作性要考虑通信、协议、接口及数据格式等因素。

4. 可管理性

可管理性是指查看和修改指定系统或软件状态的能力，它描述了系统管理员管理应用程序的难易程度。

5. *性能

性能与时间有关。性能表示的是系统的响应速度，也就是要在一定时间间隔内完成指定行为的能力。可以通过延迟和吞吐量来衡量性能，延迟是事件完成响应之前的时间，吞吐量是指一定时间内发生的事件数量。影响性能最主要的两个因素是事件源的数量和到达模式。

6. 可伸缩性

可伸缩性是一种衡量软件系统适应系统规模（用户数量、数据量、网络节点）增长的能力，目标是在负载增加情况下维护系统的可用性、可靠性和性能。

垂直扩展：在**单个服务器**上增加处理器、硬盘驱动器和 I/O 设备的数量。不需要改变底层源代码，就可以增加容量。

水平扩展：增加**更多的服务器**。会造成系统体系结构整体复杂度提高，集群维护难度大。

7. *安全性

安全性指系统防止恶意行为或系统设计的使用方式之外行为，以及防止信息泄露和丢失的能力。

2.4 系统质量属性

1. 可支持性

可支持性是系统在不正常工作情况下提供信息以确定和解决问题的能力。

涉及的关键问题包括：

1. 缺乏调试信息
2. 缺乏故障排查工具
3. 缺乏跟踪工具
4. 缺乏健康监控

2. 可测试性

软件可测试性是指通过测试揭示软件缺陷的容易程度。可测试性衡量的是为系统及其组件创建测试标准，以及执行这些测试来确定是否满足标准的要求。

2.4 用户质量属性

1. 易用性

易用性关注的是对用户来说完成某个期望任务的难易程度和系统所提供的用户支持的种类。易用性是把用户作为开发过程的中心，即“以用户为中心进行设计”。

易用性包含：

易理解性：文档、功能名称、图标、提示信息等

易学习性：文档详尽、操作一目了然

易操作性：人机界面友好、界面设计科学合理以及操作简单等。

思考：开发的系统需要满足什么质量属性及场景。

三、软件体系结构风格及案例

3.1 数据流风格

基本结构

- 连接件：数据流

- 特点：单向、通常是异步、有缓冲
- 接口角色：读 reader 和写 writer
- 计算模型：把数据从一个处理的输出端口传送到另一个处理的输入端口

1) 管道-过滤器风格

由管道、过滤器、读端口、写端口组成

- a. 构件：过滤器，处理数据流
- b. 连接件：管道，连接一个源和一个目的过滤器，转发数据流

2) 批处理

是管道过滤器风格的一种特例。

数据流风格优点：

- 1) 构件间仅通过数据传递进行交互，使得构件设计具有良好的隐蔽性和高内聚、低耦合的特点；
- 2) 支持良好的软件复用特性；
- 3) 系统维护和增强系统性能相对简单；
- 4) 每个过滤器负责完成一个单独的任务，因此可与其它任务并行执行。

缺点：

- 1) 通常导致系统进程成为批处理的结构，整体性能不高；
- 2) 不适合交互应用；
- 3) 每个过滤器都增加了解析和合成数据的工作，这样就导致了系统性能下降，并增加了编写过滤器的复杂性。

3.2 过程调用风格

1) 主程序-子过程风格

基本构成：

- 构件：主程序、子程序
- 连接器：调用-返回机制
- 拓扑结构：层次化结构

本质：将大系统分解为若干模块(模块化)，主程序调用这些模块实现完整的系统功能。

优点：

已被证明是成功的设计方法，可以被用于较大程序

缺点：

程序超过 10 万行，表现不好；程序太大，开发太慢，测试越来越困难。

2) 面向对象风格

系统被看作对象的集合；每个对象都有一个它自己的功能集合。数据及作用在数据上的操作被封装成抽象数据类型——对象。

基本构成：

- 构件是：类和对象
- 连接件：对象之间通过功能与函数调用实现交互

优点：面向对象的优点

易维护、易复用、易扩展，由于面向对象有封装、继承、多态性的特性，可以设计出低耦合的系统，使系统更加灵活、更加易于维护

缺点：需要管理大量的对象；必须知道对象的身份；继承引起复杂度，关键系统中慎用。

3) 层次风格

基本构成：

- 基本构件：各层次内部包含的构件
- 连接件：层间的交互协议
- 拓扑结构：分层
- 约束：对相邻层间交互的约束

严格分层 VS. 松散分层

严格分层

严格分层系统要求严格遵循分层原则，限制一层中的构件只能与对等实体以及与它紧邻的下面一层进行交互；第 N 层只能与第 N-1 和 N+1 层中的构件进行交互，第 N-1 层只能与第 N 和 N-2 层进行交互，依次类推。

严格分层的优点：修改时的简单性；当某一层被加入或被替换时，它的影响范围很有限，容易管理；

缺点：效率低。

松散分层

松散的分层应用程序放宽了此限制，它允许构件与位于它下面的任意层中的组件进行交互；第 N 层不仅可以与第 N-1 层交互，而且可以与第 N-2 层和第 N-3 层交互。

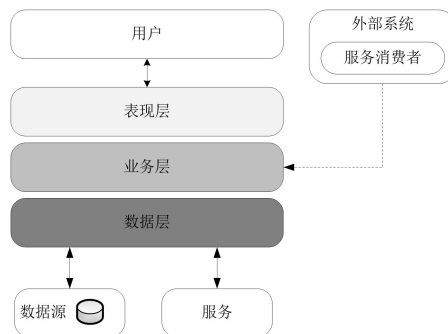
优点：松散方法可以改善效率，因为系统不必将简单调用从一层转发到下一层。

缺点：松散方法在层之间不提供相同的隔离级别，这使得在不影响较高层的情况下换出较低层变得更困难。

逻辑分层 VS. 物理分层

逻辑层

层次软件体系结构风格侧重于把软件中的相关功能和组件分成独立的逻辑分组，即逻辑层。



表现层：负责管理用户和系统之间的交互。表现层通常包含用户界面组件和表现逻辑组件。

业务层：实现了系统的核心功能并且封装了相关业务逻辑。通常包含应用程序外观和业务逻辑组件。

数据层：提供针对寄存在系统边界内的活儿其他互联网系统暴露的数据的访问。通常包含数据访问组件和服务代理。

物理层

描述如何把功能和组件从物理上部署到独立服务器、计算机、网络或远程位置。

一般情况下，一个独立运行的服务器就是一个物理层。也有几个服务器组成一个层的情况，如集群服务器就可看作是同一个层。

层次风格优点：抽象、隔离、可扩展性、可管理性、性能、可重用性、可测试性、标准化。

缺点：并不是每个系统都可以很容易地划分为分层的模式；效率的降低；难找到合适的、正确的层次抽象方法。

3.3 独立构建风格

1) 进程通信体系结构风格

构件是独立的进程，连接件是消息传递。消息传递通常用来实现进程之间的同步和对共享资源的互斥操作。

典型例子：客户-服务器架构

2) 基于事件的隐式调用风格

- 构件不直接调用一个过程，而是触发或广播一个或多个事件。
- 系统中的其它构件中的过程在一个或多个事件中注册。
- 当一个事件被触发/发布，系统自动调用在这个事件中注

册的所有过程。

d. 这样，一个事件的触发就导致了另一模块中的过程的调用。

主要特点是：

事件的触发者并不知道哪些构件会被这些事件影响，相互保持独立；

这样不能假定构件的处理顺序，甚至不知道哪些过程会被调用；

各个构件之间彼此无直接的连接关系，各自独立存在，通过对事件的发布和注册实现关联。

问题：事件管理器如何向这些过程分发相关的事件？

答案：两种策略

- 带有独立派遣模块的事件管理器。
- 没有独立派遣模块的事件管理器。

事件派遣模块如何派遣两种策略：

- 广播式(All broadcasting)：派遣模块将事件广播到所有的模块，但只有感兴趣的模块才去取事件并触发自身的行为；
- 选择广播式(Selected broadcasting)：派遣模块将事件送到那些已经注册了的模块中。包括点对点和发布-订阅模式。

优点：

- 1) 能够很好的支持交互式系统（如用户输入/网络通信等），系统中的操作可异步执行，不必同步等待执行结果。
- 2) 为软件复用提供了强大的支持。
- 3) 对事件的并发处理将提高系统性能；
- 4) 健壮性。一个构件出错将不会影响其他构件。

缺点：

- 1) 分布式的控制方式使得系统的同步、验证和调试变得异常困难。
- 2) 构件放弃了对系统计算的控制。
- 3) 不能保证过程被调用的顺序。
- 4) 传统的基于先验和后验条件的验证变得不可能。
- 5) 数据交换的问题，全局性能和资源管理便成了问题。
- 6) 隐式调用增加了中间层必要的消耗，会降低事件的响应速度。

3.4 虚拟机风格

本系统采用了 Java 语言来编写程序，

JVM：执行 Java 字节码的虚拟机，JVM 可适应所有的硬件与 OS 平台，从而使得 Java 具有“一次书写，到处运行”的能力。

虚拟机风格有两种典型的子风格，解释器风格和基于规则的系统。

解释器

解释器是一个用来执行其他程序的程序。

虚拟机风格的优点：

- 1) 可以实现非本机平台功能
- 2) 有助于软件的可移植性
- 3) 软件具有很强的灵活性
- 4) 有助于优化整个系统党的结构
- 5) 便于学习和使用。

缺点：

- 1) 执行速度慢
- 2) 额外的软件层的正确性需要进行验证
- 3) 无法最大限度地发挥操作系统和硬件的性能

3.5 客户机/服务器风格

- 胖客户端：客户端执行大部分的数据处理操作
- 瘦客户端：客户端具有很少或没有业务逻辑

胖客户端特点：

- 1) 较低的服务器需求
- 2) 离线工作
- 3) 更好的多媒体性能。
- 4) 更灵活
- 5) 充分利用已有设备资源
- 6) 更高的服务器容量

瘦客户端特点：

- 1) 单点故障
- 2) 廉价的客户端硬件
- 3) 有限的显示性能

传统两层 C/S 风格（典型的胖客户端）

优点：

由于其客户端为桌面应用程序，可开发出复杂的交互界面；此外，由于大多数业务逻辑在客户端完成，可充分利用客户端的计算能力轻松完成一些复杂计算。

缺点：

- 1) 系统伸缩性差：当用户数超过 100，性能急剧恶化，服务器成为系统的瓶颈。
- 2) 客户端需要安装专用的客户端软件，当要进行软件升级时，每台客户机都要重新安装，维护和升级成本非常高，其次,软件移植困难，新技术无法轻易使用。

三层 B/S 风格

优点：

- 1) 维护成本低：系统安装、修改和维护全在服务器端解决，维护成本低。
- 2) 安全性高：客户应用不能直接访问数据，应用服务器对数据的访问和修改具有可控性，而且可以控制数据被改变和访问的方式。
- 3) 良好的容错能力和负载均衡能力：三层模式将服务集中在一起管理，统一服务于客户端。

缺点：

- 1) 执行效率低：每次请求都要刷新一次页面。
- 2) 响应速度慢：受 http 协议限制
- 3) 数据提交一般以页面为单位，数据动态交互性不强。

三层 C/S 风格

优点：

- 1) 应用的各层可以并行开发，可以选择各自最适合的开发平台和开发语言。
- 2) 利用功能层有效地隔离开表示层与数据层，未授权的用户难以绕过功能层而非法的访问数据层，为严格的安全管理奠定了坚实的基础。
- 3) 将遗留系统移植到三层 C/S 下将非常容易；

缺点：

- 1) 三层 C/S 结构各层间的通信效率若不高。
- 2) 设计时必须慎重考虑三层间的通信方法、通信频度及数据量，这和提高各层的独立性一样 是三层 C/S 结构的关键问题。——分层风格的固有缺点

C/S+B/S 混合体系结构

遵循“内外有别”的原则：

- 1) 企业内部用户通过局域网直接访问数据库服务器(C/S 结构；交互性增强；数据查询与修改的响应速度高)

2) 企业外部用户通过 Internet 访问 Web 服务器/应用服务器 (B/S 结构; 用户不直接访问数据, 数据安全) 遵循“查改有别”的原则:

不管用户处于企业内外什么位置, 凡是需要对数据进行更新操作的(Add, Delete, Update), 都需要使用 C/S 结构; 如果只是执行一般的查询与浏览操作(Read/Query), 则使用 B/S 结构。

3.6 SOA 风格

面向服务的体系结构 (SOA) 是一个构件模型, 它将应用程序的不同功能单元通过定义良好的接口和契约联系起来。

接口是采用中立的方式进行定义的, 它应该独立于实现服务的硬件平台、操作系统和编程语言。这使得构建在各种这样的系统中的服务可以以一种统一和通用的方式进行交互。

服务 (service) 是封装成用于业务流程的可复用构件的应用程序函数。

四、软件体系结构描述与建模

4.1 常见描述方法

1. 框线图描述方法

优点: 可视化, 能够直观反映系统架构, 同时易于理解

缺点: 由于其术语和表达语义上存在着一些不规范和不精确, 从而使得以矩形为基础的传统图形表达方法在不同系统和不同文档之间存在着许多不一致甚至矛盾。体现在:

- 1) 二义性: 针对图形的本质所决定的模糊性, 不同人有不同的理解。
- 2) 矛盾性: 模型中可能存在互相冲突的陈述。
- 3) 不完备: 无法描述所有细节
- 4) 异构性: 难以支持模型在各个建模工具之间的交换
- 5) 无法自动化: 只能由人理解

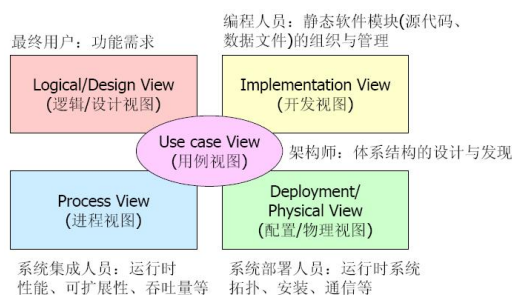
2. 形式化描述方法

能够为系统提供精确和抽象的模型, 通过这个模型分析体系结构的构成形式化方法不能直接支持软件的各种概念, 因此难以在实践中应用。

三个基本元素是: 构件、连接件、体系结构配置。

3. UML 描述方法

4.2 “4+1”视图模型

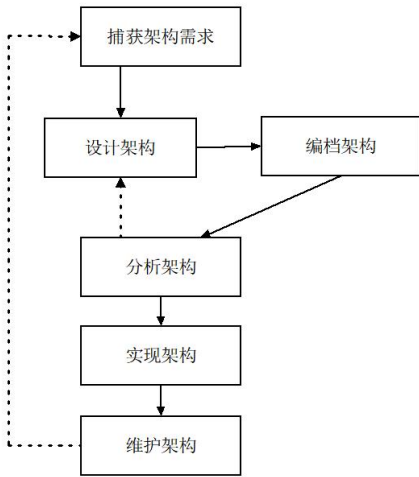


1. 逻辑视图: 类图、对象图、包图、组合图、状态图
2. 开发视图: 组件
3. 运行视图: 序列图、通信图、活动图、计时图、交互概述图
4. 部署视图: 部署图
5. 用例视图: 用例图

五、软件体系结构设计 with 评估

5.1 架构为 with 中心的软件开发过程

1. 软件体系结构生命周期模型



2. 架构中方法（ACM）与 RUP 的融合

RUP 的四个阶段：

初始阶段：主要确定项目的目标和范围

细化阶段：将焦点转移到架构风险上，主要开展项目风险、设计架构、制定项目计划等。

构建阶段：进行软件详细设计、实现和测试。

提交阶段：对系统功能、性能、质量进行完善。

5.2 属性驱动的设计方法（ADD）

1. 什么是属性驱动的设计方法？

属性驱动的设计方法是定义软件架构的一种方法，可根据软件质量属性需求实施架构设计过程。ADD 通过一个分解系统或者系统元素的循环过程，使用架构模式和策略来满足系统质量属性需求，以完成分解操作和模式

2. ADD 的设计步骤：

输入：功能需求、设计约束、质量属性需求

输出：软件架构设计

步骤一：确认有充足的需求信息

首先需要搜集足够多的系统需求，并将其作为 ADD 的输入；还需捕获足够多的系统质量属性需求并将其描述成具体可度量的质量属性响应（刺激源、刺激、制品、环境、响应、响应度量）

步骤二：选择系统一个元素进行分解

在步骤 2 中，将所选择的系统元素作为后续步骤的设计重点。当首次执行该步骤时，唯一可以分解的元素是系统本身（大泥球），那么就可将所有需求分配给该系统；已经将系统分为两个或者多个元素，也将需求分配给这些元素。

步骤三：识别候选架构驱动

根据需求对架构的影响程度，再次将这些需求进行排序。

步骤四：选择一个满足架构驱动的设计构想

需要选出架构中的主要元素及其之间的关系。根据设计约束和质量属性需求确定元素、元素间关系及其之间的交互机制。

步骤五：实例化架构元素并分配职责

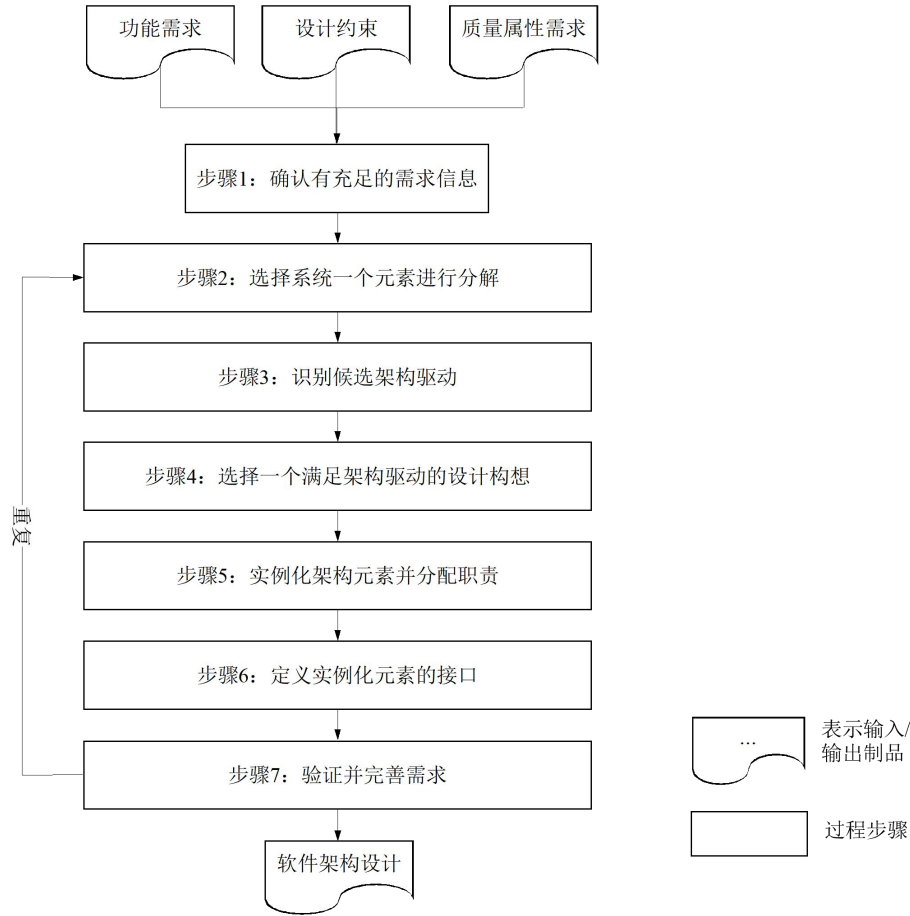
实例化在之前步骤中选择的软件元素，根据元素的类型为它们分配职责；实例化的元素的职责也来源于候选架构驱动关联的功能性需求以及其父元素关联的功能性需求

步骤六：定义实例化元素的接口

- 定义元素所需提供的服务和性质。
- 一个接口可能包括以下任何一项：
 - 操作语法（如签名）
 - 操作语义（如描述，前置条件，后置条件，约束）
 - 信息交换（如信号事件，全局数据）
 - 个体元素或操作的质量属性需求
 - 错误处理

步骤七：验证并完善需求

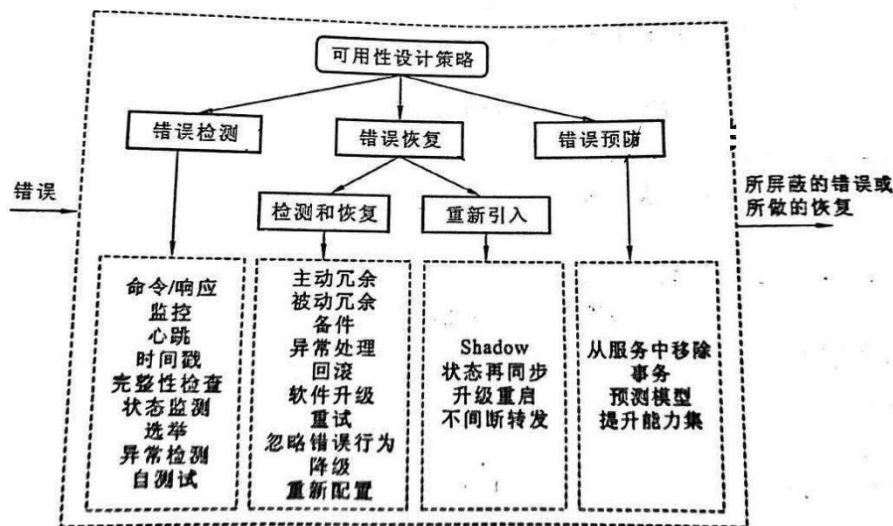
检验元素分解是否满足功能性需求、质量属性需求以及设计约束。同时，准备子元素为接下来的分解做准备。



5.3 常见质量属性设计策略

1. 可用性设计策略

阻止错误发展成故障，至少能把错误的影响限制在一定范围之内，从而使得系统修复成为可能。



1) 错误检测

- 命令/响应：一个构件发出一个命令，测试在预定时间内收到一个来自审查构件的响应。
- 心跳：一个构件定期发出一个心跳消息，另一个构件收听该消息，如果心跳失败通知错误处理构件；自动柜员机可以定期向服务器发送上一次交易的日志，起到心跳和传送数据双重作用。

2) 错误恢复

包括检测和恢复、重新引入。

检测和恢复：

- 异常处理：系统以某种方式处理检测到的异常。
- 回滚：允许系统能够恢复到之前的一个正常状态。

重新引入：

- Shadow 操作，设置一个构件短时间内以“shadow 模式”运行，以确保在恢复该构件之前，模仿工作构件的行为。
- 升级重启：通过改变重启组件的粒度并最小化所影响服务的级别来使系统同从错误中恢复。

3) 错误预防

- 事务：绑定几个有序的步骤，在发生故障时能够立即撤销整个绑定。
- 预测模型：该策略指的是应用一定的技术来阻止系统发生异常。如：使用异常类来使系统从异常中透明地恢复。

2. *性能设计策略

性能设计策略的目标是当某一事件到达系统后，系统能在一定的时间约束内做出响应。

1) 控制资源需求

- 提高计算效率：改进在关键的地方所使用的算法的效率将减少等待时间。
- 划分事件优先级：所有的事件并非同等重要，就可以根据每个事件的重要程度进行排序并赋予优先级。

2) 管理资源

a. 引入并发

如果可以并行进行请求，可以减少闭锁时间。通过在不同的线程上处理不同的事件流或创建额外的线程来处理不同的活动集来引入并发。

b. 增加可用资源

速度更快的处理器、额外的处理器、额外的内存以及速度更快的网络都可以减少等待时间。

3. *安全性设计策略

考虑采用什么方法让系统达到安全性时可以联想到实际的物理安全控制。

1) 检测攻击

通过“入侵检测”系统进行，将通信模式和已知攻击的历史模式进行比较

2) 抵御攻击

- a. 验证参与者：确保参与者与其所声称的身份保持一致。如登录时的验证码
- b. 用户授权：保证经过了身份认证的用户有权访问和修改数据或者服务。

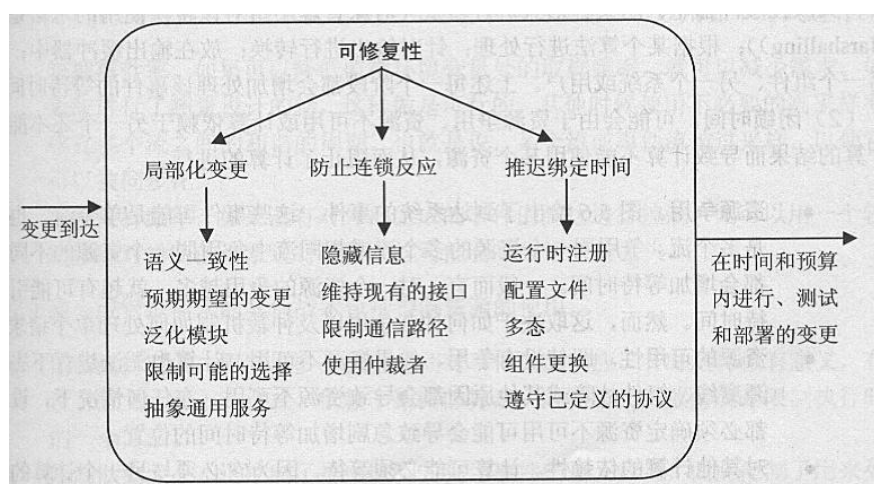
3) 攻击反应

- a. 锁定计算机：多次输入错误登录可能预示着有潜在的攻击。系统可以采取对多次输入错误的账号进行锁定。
- b. 通知参与者：在遭受到持续攻击时往往需要一些人员或系统操作，这些人员或系统操作必须在系统检测到攻击时得到通知。

4) 从攻击中恢复

- a. 恢复状态：可以采用可用性设计策略，都是从不一致的状态恢复到一致状态。
- b. 识别攻击者：使用审计信息来追踪攻击者的操作。

4. 可修改性



信息隐蔽：把某个实体的责任分解为更小的部分，并选择哪些信息是公有的，哪些信息是私有的。

维持现有的接口

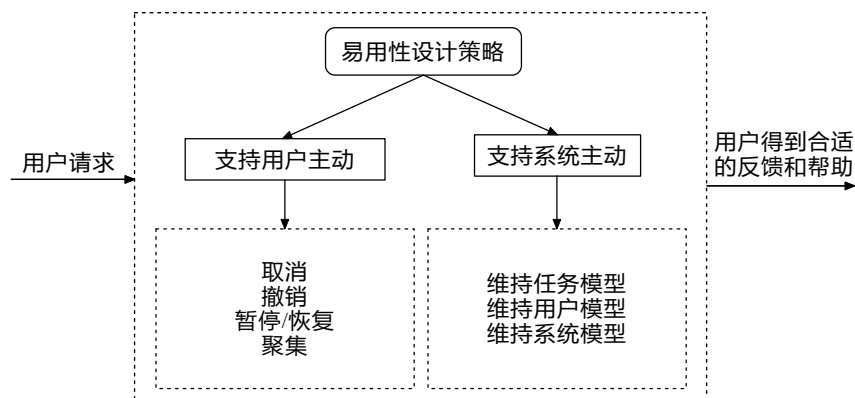
添加接口：通过添加新的接口提供新增的可见服务和数据，从而使得现有的接口不变

运行时注册：支持即插即用操作（插件），但需要管理注册的额外开销

多态：允许方法调用的后期绑定。

5. *易用性

易用性关注的是用户如何能够简单的完成一项任务，以及系统能够提供给用户的支持。



1) 支持用户主动

撤销：与取消不同，撤销操作需要系统回到之前的某个状态。如不小心删除了某个文件，可通过系统在关键点的“快照”进行恢复。

暂停/恢复：当一个用户启动一个长时间的操作，如从服务器下载较大文件，为用户提供暂停和恢复操作有助于提高易用性。

聚集：当一个用户正在执行重复的操作，或者该操作对许多对象产生了相同的影响时，将低级别的对象聚集到一个小组中，然后将操作应用到整个小组。如：用户批量上传、删除文件。

2) 支持系统主动

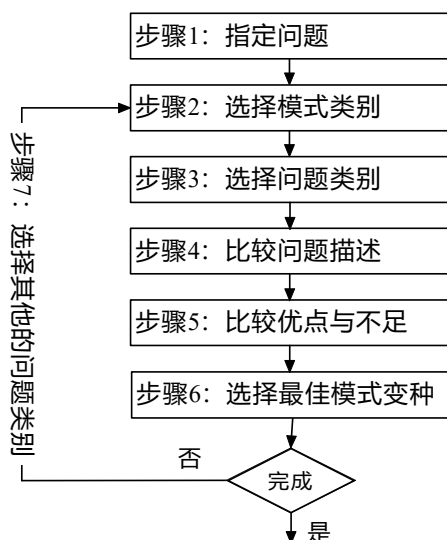
维持任务模型：任务模型用来判断上下文，使得系统能够知道用户正在做什么并对用户提供帮助。

维持系统模型：系统自身维持一个模型，该模型用来决定期望的系统行为，使其能向用户传递合适的反馈信息。一个系统模型的常见示例是进度条，它能够预测完成当前活动所需的剩余时间。如：用户在使用网盘上传文件时，会展示上传文件的进度条，显示当前上传的进度。预测当前上传所需的时间。

5.4 基于模式的设计方法

1. 设计步骤

- 1) 选择一个满足架构驱动的设计构想
- 2) 为该构件定义需求
- 3) 针对步骤 2 中定义的需求和交互，找出最适合的架构风格或模式



- 4) 使用与问题相匹配的模式来指导类和构件的设计
- 5) 在构件中进行迭代，为每个构件重复步骤 2-4

5.5 模块化设计的基本原则

1. 类设计原则

- 1) 单一责任原则：一个类，只有一个引起它变化的原因。应该只有一个职责。
- 2) 开放-封闭原则：对扩展是开放的，而对修改是封闭的。
- 3) 里氏替换原则：子类可以扩展父类的功能，但不能改变父类原有的功能。
 - ① 子类可以实现父类的抽象方法，但是不能覆盖父类的非抽象方法。
 - ② 子类中可以增加自己特有的方法。
 - ③ 当子类覆盖或实现父类的方法时，方法的前置条件（即方法的形参）要比父类方法的输入参数更宽松。
 - ④ 当子类的方法实现父类的抽象方法时，方法的后置条件（即方法的返回值）要比父类更严格。
- 4) 依赖倒置原则：高层模块，低层模块，细节都应该依赖抽象
- 5) 接口隔离原则：客户端不应该依赖它不需要的接口；

2. 包聚合设计原则

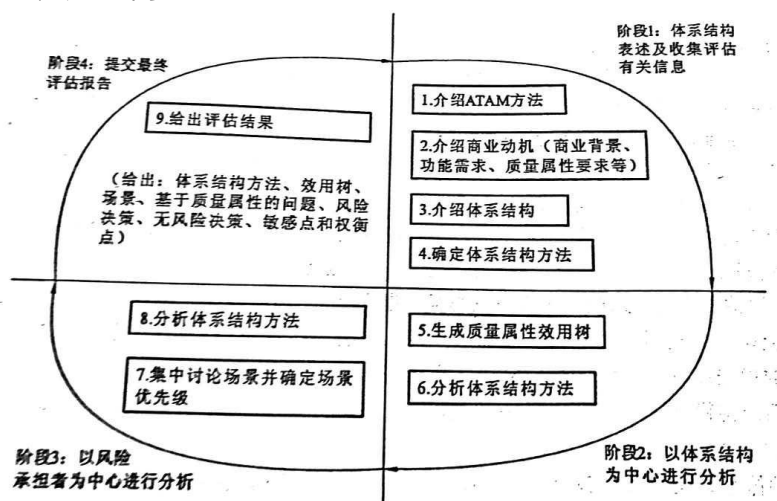
- 1) 复用/发布等价原则
- 2) 共同封闭原则
- 3) 共同复用原则

3. 包耦合设计原则

- 1) 无圈依赖原则
消除圈（循环依赖）的两种方式
 - ① 创建新包
 - ② 使用依赖倒置原则
- 2) 稳定依赖原则
- 3) 稳定抽象原则

5.6 体系结构评审方法

ATAM 方法包括四大部分，共九个步骤



- 1) 描述 ATAM 方法：评估小组负责人向参加会议的风险承担者介绍 ATAM 评估方法。
- 2) 描述商业动机：参加评估的所有人员必须理解待评估的系统。
- 3) 描述体系结构：架构设计师要对软件体系结构做详细适当的讲解，这种信息讲解
- 4) 确定体系结构方法：ATAM 评估方法主要通过理解体系结构设计方法来分析体系结构，在这一步，由设计师确定体系结构设计方法。
- 5) 生成质量属性效用树：在这一步，评估小组、管理人员和客户代表一起确定系统最重要的质量属性目标，并对这些质量目标设置优先级和细化。
- 6) 分析体系结构方法：一旦得到了效应树的结果，评估小组对实现重要质量属性的体系结构方法进行考察。
- 7) 讨论和分级场景
- 8) 分析体系结构方法（是第六步的重复）
- 9) 描述评估结果：最后，要把 ATAM 分析中所得到的各种信息进行归纳，并反馈给风险承担者（开发人员、维护人员、操作人员、终端人员、中间商、测试人员、系统管理人员等所有与系统有关的人员）。
质量属性冲突时怎么办？